

# Aufgaben - Reverse Engineering

**Dozent:** [Prof. Dr. Michael Eichberg](#)  
**Kontakt:** [michael.eichberg@dhbw-mannheim.de](mailto:michael.eichberg@dhbw-mannheim.de), Raum 149B  
**Version:** 1.0.0

# Einführung

Ziel dieser Aufgabe ist es einen ersten Einblick in den Bereich des Software Reverse Engineering zu bekommen. Da es beim Reverse Engineering auch darum gehen kann Schutzsysteme zu analysieren - z. B. durch das Aufdecken von Schwachstellen oder durch Brute-Force - ist dies auch in diesem Fall Ihre Aufgabe.

Für diese Aufgaben ist es ggf. notwendig (ein bisschen) Code zu schreiben.

Sie können diesen zum Beispiel in folgenden Sprachen schreiben: Java (ggf. als Skript), Python, Scala, C, C++, Rust, JavaScript, TypeScript, Prolog, Bash oder Zsh. Es ist oft der Fall, dass der Code, der im Rahmen des Reverse Engineering entwickelt wird, eine andere Sprache nutzt als der Originalcode.

Es ist im Rahmen des Reverse Engineering fast immer so, dass mehrere Wege zum Ziel führen. Sollte ein Weg nicht erfolgreich sein, dann probieren Sie einen anderen. Das Wichtigste ist Beharrlichkeit!

Es steht ihnen vollkommen frei welchen Weg sie gehen.

## !! Wichtig

KIs sind in der Zwischenzeit sehr mächtig und hilfreich!

## ※ Hinweis

Für die Analyse der nativen Binärdateien können Sie gerne Werkzeuge wie Ghidra, Radare2 oder ähnliche verwenden. Ggf. ist der Einsatz von Kali Linux hilfreich. Ghidra kann dort einfach mit `sudo apt install ghidra` installiert werden. Die Binaries wurden alle in einer VM getestet und laufen dort problemlos. Ggf. ist die Emulation (!) von ARM64 oder x86\_64 erforderlich.

## ※ Hinweis

Sollten Sie Probleme haben die nativen Binärdateien auszuführen, dann nutzen Sie ggf. das Tool `file` um herauszufinden, für welche Architektur die entsprechende Binärdatei kompiliert wurde und welche Bibliotheken ggf. fehlen bzw. installiert werden müssen.

(Es gibt zum Beispiel verschiedene Implementationen der C Standardbibliothek. Diese können sich in Details unterscheiden, was dazu führen kann, dass eine Binärdatei nicht ausgeführt werden kann und es dann zu - teilweise sehr verwirrenden - Fehlermeldungen kommt.)

## ※ Hinweis

Die Analyse von Java Class Dateien kann zum Beispiel über FenFlower (eingebaut in IntelliJ IDEA; als Plug-in für VS Code verfügbar) oder CFR erfolgen.



# EncryptMe

## 0.1. Mit EncryptMe verschlüsselte Datei entschlüsseln

Ziel dieser Aufgabe ist es die Datei zu entschlüsseln. Die Verschlüsselung ist mit der Software `encryptme` durchgeführt worden.

[Encryptme.zip](#)

### ※ Hinweis

Die Software liegt sowohl als ARM als auch als x86\_64 Version vor. Beide sind in der Lage die Datei zu entschlüsseln und funktional identisch. Sollten Sie also mit einer der Architekturen vertrauter sein, dann verwenden Sie diese.

## EncryptMe2

### 0.2. Mit EncryptMe2 verschlüsselte Datei entschlüsseln

Ziel dieser Aufgabe ist es die Datei zu entschlüsseln. Die Verschlüsselung ist mit der Software `encryptme2` durchgeführt worden.

[Encryptme2.zip](#)

#### ※ Hinweis

Die Software liegt sowohl als ARM als auch als x86\_64 Version vor. Beide sind in der Lage die Datei zu entschlüsseln und funktional identisch. Sollten Sie also mit einer der Architekturen vertrauter sein, dann verwenden Sie diese.



## Secure24v1

Ihnen liegt eine verschlüsselte Datei vor: `Message.odp.encrypted`, die mit dem Program `Secure24v1` verschlüsselt wurde. Informationen zum verwendeten Passwort liegen nur begrenzt vor. Es ist davon auszugehen, dass das Passwort sicher ist. Es liegt Ihnen aber noch die Datei `key` vor. Weiterhin ist davon auszugehen, dass die verschlüsselte Datei tatsächlich eine OpenOffice/LibreOffice Präsentation ist.

Analysieren Sie die Anwendung, um einen möglichen Ansatzpunkt zu finden, um die Datei erfolgreich zu entschlüsseln.



### Hintergrund

Informationen über den Aufbau von OpenOffice Dateien finden sich zum Beispiel hier:

<http://docs.oasis-open.org/office/v1.0/OpenDocument-v1.0-os.pdf>

Insbesondere Abschnitt 17 enthält Informationen zum Aufbau von entsprechenden Dateien und wie diese erkannt werden können.

[Secure24v1.zip](#)



## Secure24v2

Neben dem Programm `secure24v2` liegen die beiden Dateien: `Text.encrypted` und `key` vor. In diesem Falle sind keinerlei Informationen über das Passwort bekannt. Beim Inhalt ist davon auszugehen, dass es sich um eine einfache Textdatei handelt. Ziel ist die Entschlüsselung der Datei.

[Secure24v2.zip](#)

